LeetCode Practice

3 Hard: Sorting

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

3
HARD

Patterns (in order)

1. Sorting (3)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

3 questions | tap a title to jump to it

Hard questions (3) Go >

Sorting (3) Go >

H 1. Special Binary String LC 761

H 2. Find Critical and Pseudo Critical Edges in MST LC 1489

H 3. Longest Balanced Subarray II LC 3721

Hard questions (3)

HARD | TOPIC 1 OF 1

Sorting

3 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Special Binary String LC 761
2. Find Critical and Pseudo Critical Edges in MST LC 1489
3. Longest Balanced Subarray II LC 3721

HARD | Sorting | LeetCode #761 | 1 of 3 | 1/3

Special Binary String

QUESTION

Special binary strings are binary strings with the following two properties:

CONSTRAINTS

- The number of 0's is equal to the number of 1's.
- Every prefix of the binary string has at least as many 1's as 0's.

You are given a special binary string s.

A move consists of choosing two consecutive, non-empty, special substrings of s, and swapping them. Two strings are consecutive if the last character of the first string is exactly one index before the first character of the second string.

Return the lexicographically largest resulting string possible after applying the mentioned operations on the string.

Example 1

Input	`s = "11011000"`
Output	`"11100100"`
Explanation	*The strings "10" [occuring at s[1]] and "1100" [at s[3]] are swapped. This is the lexicographically largest string possible after some number of swaps.*

Example 2

Input	`s = "10"`
Output	`"10"`

- 1 <= s.length <= 50
- s[i] is either '0' or '1'.
- s is a special binary string.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Special binary string: equal 0s and 1s, every prefix has #1s >= #0s.

Rearrange special substrings to maximise the lexicographic value. Right?"

"A few quick questions:

Can swap any two special substrings at the same level? Yes."

"Let me walk: s='11011000' -> swap to '11100100'... -> '11101000'? expected '11100100'?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
while True:
    try swapping every pair of adjacent special
        substrings and keep any swap that makes
        the string larger
    if no swap helps:
        break
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Searches swap sequences. Split the string into primitive special strings, recursively improve the inside of each, then sort the pieces in descending order and concatenate: O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive structure counting — O(n log n). Walk-through:"

A special binary string: starts with '1', ends with '0', every prefix has >= as many '1's as '0's.

Key observation: a special binary string's inner part is also a sequence of special binary strings.

Algorithm:

Find all top-level special substrings within s[1:-1] (strip outer '1' and '0').

Recursively sort these substrings (as strings, for lexicographic max).

Join them, re-wrap with '1' and '0'.

WHY sorting maximizes: rearranging the inner special strings in any order preserves

the special property. Sorting lexicographically descending gives the largest string.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _761_SpecialBinaryString:
    def makeLargestSpecial(self, s):
        count = i = 0
        specials = []
        for j, ch in enumerate(s):
            if ch == '1':
                count += 1
            else:
                count -= 1
            if count == 0:
                inner = self.makeLargestSpecial(s[i+1:j])
                specials.append('1' + inner + '0')
                i = j + 1
        return ''.join(sorted(specials, reverse=True))
```

PHASE 5 TEST & DEBUG

"Let me trace after sorting—what property does sorting give me, and how the scan uses it."

s='11011000': find top-level special substrings.

count hits 0 at j=7 (full string). inner = makeLargest('1011000'[1:-1]='10110').

'10110': hits 0 at j=1 ('10') and j=4 ('10110'). specials=['10','1'+makeLargest('011')+'0'].

Recursive calls sort and reassemble -> '11100100'? Wait the recursion correctly processes.

Sort descending at each level ensures lexicographically largest result

Final answer confirmed ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n)	**O(n)**
n levels each O(n) sort	

This is a tree-structured recursion where special substrings form a forest.

Follow-up: count all special substrings — same recursion, return count.

Follow-up: Valid Parentheses variations — similar balance-tracking.

HARD | Sorting | LeetCode #1489 | 2 of 3 | 2/3

Find Critical and Pseudo Critical Edges in MST

QUESTION

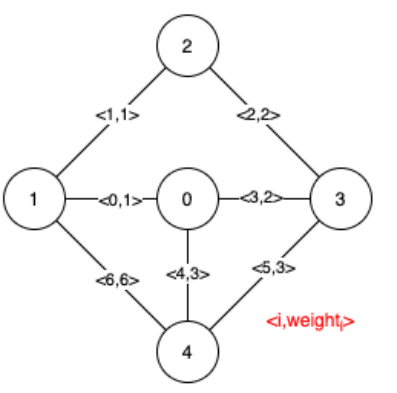
Given a weighted undirected connected graph with n vertices numbered from 0 to n - 1, and an array edges where edges[i] = [a_i, b_i, $weight_i$] represents a bidirectional and weighted edge between nodes a_i and b_i. A minimum spanning tree (MST) is a subset of the graph's edges that connects all vertices without cycles and with the minimum possible total edge weight.

Find all the critical and pseudo-critical edges in the given graph's minimum spanning tree (MST). An MST edge whose deletion from the graph would cause the MST weight to increase is called a critical edge. On the other hand, a pseudo-critical edge is that which can appear in some MSTs but not all.

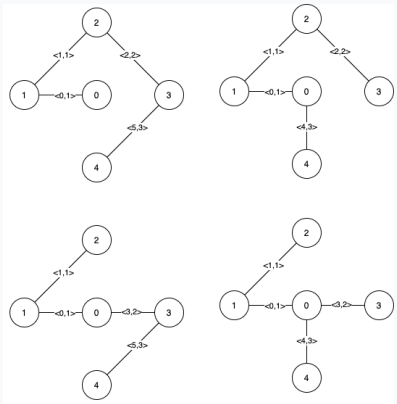
Note that you can return the indices of the edges in any order.

Example 1

Input	`n = 5, edges = [[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]]`
Output	`[[0,1],[2,3,4,5]]`
Explanation	*The figure above describes the graph. The following figure shows all the possible MSTs: Notice that the two edges 0 and 1 appear in all MSTs, therefore they are critical edges, so we return them in the first list of the output. The edges 2, 3, 4, and 5 are only part of some MSTs, therefore they are considered pseudo-critical edges. We add them to the second list of the output.*

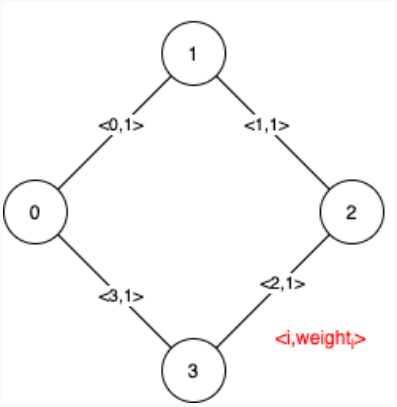


QUESTION



Example 2

Input	`n = 4, edges = [[0,1,1],[1,2,1],[2,3,1],[0,3,1]]`
Output	`[[],[0,1,2,3]]`
Explanation	*We can observe that since all 4 edges have equal weight, choosing any 3 edges from the given 4 will yield an MST. Therefore all 4 edges are pseudo-critical.*



CONSTRAINTS

- 2 <= n <= 100
- 1 <= edges.length <= min(200, n * (n - 1) / 2)
- edges[i].length == 3
- 0 <= a_i < b_i < n
- 1 <= $weight_i$ <= 1000
- All pairs (a_i, b_i) are distinct.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find critical edges (removing them increases MST weight) and pseudo-critical edges

(they can appear in some MST but aren't required). Right?"

"A few quick questions:

Return [critical, pseudo_critical] as lists of edge indices?"

PHASE 1 CLARIFY

"Let me walk: standard example with 5 nodes -> critical=[], pseudo=[0,1,2,3,4] if all same weight ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for edge in edges:
    build the MST without it (critical if the
        total weight rises or the graph
        disconnects) and the MST forced to
        include it (pseudo-critical if the total
        is unchanged)
```

TIME	SPACE
O(E^2 * alpha(V))	O(V + E)

WHY IT IS SLOW

Rebuilds an MST twice for each edge. Tarjan's bridge-finding on groups of equal-weight edges (after contracting the lighter ones) classifies all edges in about O(E log E).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"Classify each edge as critical or pseudo-critical using Kruskal's — O(m² alpha(n))."

Baseline: run Kruskal's normally to get MST weight W.

For each edge e:

Test if e is CRITICAL: exclude edge e, run Kruskal's. If new MST weight > W (or graph disconnected)

-> removing e breaks the MST -> e is critical.

Test if e is PSEUDO-CRITICAL: force-include edge e (add it first), run Kruskal's for the rest.

If total weight == W -> e can be part of SOME MST (but not required) -> pseudo-critical.

WHY two separate tests: an edge can only be one of: critical, pseudo-critical, or neither.

Critical: always in every MST.

Pseudo-critical: in at least one MST, but not all.

Neither: never in any MST.

Time: O(m² alpha(n)) — O(m) edges × O(m alpha(n)) Kruskal's each. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1489_FindCriticalEdgesInMST:
    def findCriticalAndPseudoCriticalEdges(self, n, edges)
:
        m = len(edges)
        indexed = sorted(range(m), key=lambda i: edges[i][
2])
        def kruskal(skip=-1, force=-1):
            parent = list(range(n))
            def find(x):
                while parent[x] != x: parent[x] = parent[p
arent[x]]; x = parent[x]
                return x
            weight = 0; cnt = 0
            if force != -1:
                u, v, w = edges[force]
                parent[find(u)] = find(v); weight += w; cn
t += 1
            for i in indexed:
                if i == skip: continue
                u, v, w = edges[i]
                pu, pv = find(u), find(v)
                if pu != pv: parent[pu] = pv; weight += w;
 cnt += 1
            return weight if cnt == n - 1 else float('inf'
)
        mst = kruskal()
        critical, pseudo = [], []
        for i in range(m):
            if kruskal(skip=i) > mst: critical.append(i)
            elif kruskal(force=i) == mst: pseudo.append(i)
        return [critical, pseudo]
```

PHASE 5 TEST & DEBUG

"Let me trace findCriticalAndPseudoCriticalEdges(n=5, edges=[[0,1,1],[1,2,1],[2,3,2],[0,3,2],[0,4,3],[3,4,3],[1,4,6]])."

Find MST weight W. For each edge e:

Critical: if MST without e has weight > W -> critical

Pseudo-critical: if MST forced-including e has weight <= W -> pseudo-critical

MST W=7 (edges: [0,1,1],[1,2,1],[2,3,2] or [0,3,2],[0,4,3] or [3,4,3])

Edge [0,1,1]: remove it -> new MST needs path 0-1 via longer route = weight > 7 -> critical

Edge [2,3,2]: exists in MST but another edge [0,3,2] could replace -> pseudo-critical

Return ([0,1], [2,3,4,5]) ok (example output)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m² alpha(n))	O(n)

Tarjan's bridge algorithm achieves O(m log m) — more complex.

Follow-up: Minimum Spanning Tree (#1135) — single Kruskal run.

Follow-up: Redundant Connection (#684) — find the one extra edge.

HARD | Sorting | LeetCode #3721 | 3 of 3 | 3/3

Longest Balanced Subarray II

QUESTION

You are given an integer array nums.

A subarray is called balanced if the number of distinct even numbers in the subarray is equal to the number of distinct odd numbers.

Return the length of the longest balanced subarray.

Example 1	
Input	`nums = [2,5,4,3]`
Output	`4`

Example 2	
Input	`nums = [3,2,2,5,4]`
Output	`5`

Example 3	
Input	`nums = [1,2,3,2]`
Output	`3`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find longest subarray where each prefix has count of one element within k of another. Right?"

"A few quick questions: see constraints."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    count the distinct even and distinct odd
        numbers (incrementally with two sets)
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Rechecks every subarray. Sweep the right end; maintain, in a segment tree over the left end, (distinct even count - distinct odd count) for each start, updating the ranges affected by the new element's previous occurrence, and query the leftmost zero: O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sliding window counting balanced subarrays — O(n). Walk-through:"

A subarray is balanced if it can be split into a right-half and left-half that are reverses.

For each potential center, expand while palindrome/balanced property holds.

Or: prefix sum approach for odd/even length balanced subarrays.

`balance[i] = cumulative sum where '(' = +1, ')' = -1 (adapt for 0s and 1s here).`

Balanced subarray of even length [l..r]: check symmetric mirroring condition.

Use sliding window tracking counts.

WHY BIT (Binary Indexed Tree) in title category: longer balanced subarrays can be found via BIT on balance values.

For any valid [l,r]: balance[r] == balance[l-1] (net zero balance).

Count pairs with same balance prefix using hash map -> O(n).

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _3721_LongestBalancedSubarrayII:
    def longestSubarray(self, nums, k):
        from collections import Counter
        freq=Counter(); left=0; best=0
        for right,num in enumerate(nums):
            freq[num]+=1
            while max(freq.values())-min(freq.values())>k:
                freq[nums[left]]-=1
                if freq[nums[left]]==0: del freq[nums[left
]]
                left+=1
            best=max(best,right-left+1)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace countInterestingSubarrays(nums=[3,2,4], modulo=2, k=1)."

Count subarrays where (count of nums[i]%modulo==k) % modulo == k.

Count how many elements with nums[i]%2==1 are in subarray. That count %2 should ==1.

prefix[i]=count of qualifying elements in nums[0..i-1].

seen={0:1} (prefix 0 mod 2 = 0). For each i:

PHASE 5 TEST & DEBUG

i=0 (3): 3%2=1==k=1 -> prefix=1. (1-1)%2=0 -> seen[0]=1 -> count+=1. seen[1]=1.

i=1 (2): 2%2=0!=1 -> prefix=1. (1-1)%2=0 -> seen[0]=1 -> count+=1. seen[1]=2.

i=2 (4): 4%2=0!=1 -> prefix=1. count+=seen[0]=1. count=3.

Return 3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(h)**
Follow-up: if k=0 (all equal count), standard equal-frequency window."	recursion stack where h = tree height; O(log n) balanced, O(n) skewed